

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science **ASSESSMENT**

TOOL 3 – Comparative Analysis

Course Code:	DSA0305	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	P. KALYANI	Reg. No.:	192472199
Section / Batch:	CSE-AI	Date:	

Code of Conduct

I **P. KALYANI (192472199)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. Kalyani

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3

Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. CFG Trees vs Dependency Parsing Structures

Introduction

A language processing system needs to represent the structure of sentences for grammar checking, syntactic analysis, and relationship extraction. Two important approaches are Context-Free Grammar (CFG) trees and dependency parsing.

A CFG represents a sentence using hierarchical constituents, whereas dependency parsing represents relationships between individual words. Therefore, the choice depends on whether the system primarily needs phrase structure or word-to-word relationships.

1.1 Context-Free Grammar (CFG) Trees

A CFG represents a sentence using grammatical rules such as:

$S \rightarrow NP VP$

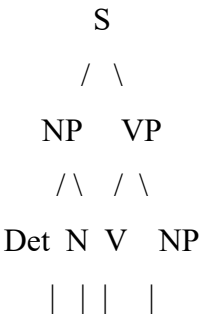
$NP \rightarrow Det N$

$VP \rightarrow V NP$

For example:

The student reads books.

A simplified CFG structure is:



The student reads books

CFG is useful for identifying:

- Sentence

- Noun phrase
- Verb phrase
- Subject
- Object
- Phrase boundaries

Advantages

- Provides clear hierarchical sentence structure.
- Useful for grammar checking.
- Represents constituent relationships effectively.
- Easy to interpret for traditional linguistic analysis.
- Provides a foundation for syntactic parsing.

Limitations

CFG can become complex for sentences with:

- long-distance relationships
- free word order
- complex modifiers
- multiple clauses
- discontinuous dependencies

1.2 Dependency Parsing

Dependency parsing focuses on relationships between individual words rather than grouping words into phrases.

For:

The student reads books.

The dependency relationships can be represented as:

reads

|— student → subject

└— books → object

The dependency representation identifies:

- student → subject of reads
- books → object of reads

Thus, the central element is generally the head word, with other words connected to it through grammatical dependencies.

1.3 Comparison

Aspect	CFG Trees	Dependency Parsing
Basic representation	Constituents/phrases	Word-to-word dependencies
Main focus	Phrase structure	Grammatical relationships
Structure	Hierarchical tree	Dependency tree/graph
Subject identification	Through NP structure	Direct subject dependency
Object identification	Through VP/NP structure	Direct object dependency
Long-distance relationships	Can become complex	Generally easier to represent
Grammar checking	Very useful	Very useful
Relation extraction	Requires additional processing	Directly useful
Interpretability	High	High
NLP applications	Grammar and constituency analysis	Information extraction and semantic analysis

Critical Analysis

CFG is better when the objective is to understand how words form larger grammatical phrases.

Dependency parsing is more effective when the objective is to determine how words are grammatically related to each other.

For example:

The doctor prescribed medicine to the patient.

Dependency parsing can directly identify:

doctor → subject → prescribed

medicine → object → prescribed

patient → indirect/oblique argument → prescribed

This makes dependency parsing particularly useful for information extraction.

Justification

For the stated requirement of capturing relationships between words, dependency parsing is more effective.

CFG should still be retained when detailed constituent structure is required.

Conclusion

CFG → better for phrase and hierarchical structure

Dependency Parsing → better for direct word relationships

Therefore, for a system specifically focused on relationships between words, dependency parsing is the preferred approach.

2. Top-Down Parsing vs Earley Parsing

Introduction

A real-time NLP system must handle:

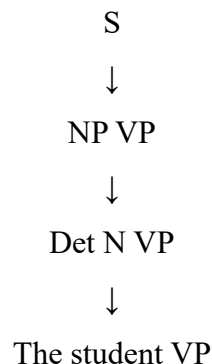
- incomplete sentences
- ambiguous input
- long sentences
- rapidly changing user input
- response-time constraints

Top-down parsing and Earley parsing approach this problem differently.

2.1 Top-Down Parsing

Top-down parsing begins with the start symbol and attempts to generate the input.

For example:



The parser continues expanding grammar rules until it matches the input.

Advantages

- Conceptually simple.
- Easy to implement for small grammars.
- Works well for relatively simple and predictable inputs.
- Can be efficient when the grammar is small and unambiguous.

Limitations

1. Backtracking

When one selected rule fails, the parser may need to return to an earlier decision and try another rule.

2. Ambiguity

Multiple possible structures can result in repeated exploration.

3. Incomplete input

A partially spoken or typed sentence may not immediately produce a complete parse.

4. Large grammars

The number of possible rule expansions can increase significantly.

2.2 Earley Parsing

Earley parsing is a chart-based parsing algorithm capable of handling general context-free grammars.

It uses three major operations:

Prediction

Predicts possible grammar rules.

Scanning

Matches the next input token.

Completion

Completes a grammar rule when its required components have been recognized.

The parser stores intermediate states in a chart, reducing unnecessary repeated work.

2.3 Comparison

Aspect	Top-Down Parsing	Earley Parsing
Starting point	Start symbol	Start symbol with chart states
Backtracking	Can be significant	Reduced through chart parsing
Ambiguity	More difficult	Handles ambiguity naturally
Incomplete input	Limited	Better suited
Left recursion	Can cause problems	Supported
General CFG	Limited depending on implementation	Supports general CFG
Memory	Usually lower	Higher due to chart
Implementation	Simpler	More complex
Dynamic input	Less suitable	More suitable
Real-time conversational NLP	Limited	Better suited

2.4 Example

Consider:

“Book a flight to Delhi...”

The user may pause before completing the command.

A top-down parser may be unable to determine the complete final structure until additional input arrives.

Earley parsing can maintain partial states and continue parsing when additional words are received.

For example:

Book → action

flight → object

Delhi → destination

The parser can preserve the partially recognized structure.

2.5 Critical Analysis

Top-down parsing is attractive because it is simple and may be fast for uncomplicated grammars.

However, real-time applications such as:

- voice assistants
- chatbots
- interactive search
- conversational systems

frequently receive incomplete or ambiguous input.

Therefore, robustness is more important than the simplicity of the parser.

Justification

Earley parsing is more suitable for dynamic input **conditions** because it can handle ambiguity, partial structures and general CFGs without relying heavily on repeated backtracking.

Conclusion

For simple and controlled applications, top-down parsing may be sufficient.

For a dynamic real-time NLP application, Earley parsing is the better choice because of its ability to maintain partial parsing states and handle ambiguous input.

3. CFG vs PCFG vs Neural Parsing for Ambiguous Sentences

Problem

Consider:

“She saw the man with a telescope.”

The sentence has an important structural ambiguity.

Interpretation 1

She used a telescope to see the man.

She saw [the man] [with a telescope].

Here, “with a telescope” modifies the action saw.

Interpretation 2

The man had a telescope.

She saw [the man with a telescope].

Here, “with a telescope” modifies the man.

Thus, the same sentence has multiple valid interpretations.

3.1 CFG

CFG can generate multiple valid parse trees.

For example:

$S \rightarrow NP VP$

$VP \rightarrow V NP$

$VP \rightarrow VP PP$

$NP \rightarrow NP PP$

The grammar can produce both interpretations.

Advantage

CFG clearly represents the possible syntactic structures.

Limitation

A basic CFG does not inherently know which interpretation is more likely.

Therefore:

CFG can represent ambiguity but does not effectively resolve it.

3.2 PCFG

A Probabilistic Context-Free Grammar associates probabilities with grammar rules.

For example:

$VP \rightarrow VP PP$ [0.60]

$NP \rightarrow NP PP$ [0.40]

The parser calculates the probability of alternative parse trees and chooses the highest-probability interpretation.

Advantage

PCFG provides a principled way to rank competing syntactic structures.

Limitation

The probabilities depend heavily on training data and grammar design.

PCFG may also have difficulty incorporating rich contextual information.

3.3 Neural Parsing

Neural parsers use machine-learning models to learn syntactic patterns from large datasets.

Modern neural systems can use contextual representations to understand how surrounding words influence meaning.

For example:

“She saw the man **with a telescope**.”

The model can use:

- surrounding words
- sentence context
- learned linguistic patterns
- contextual word representations

to estimate the more likely interpretation.

Advantages

- Strong contextual understanding.
- Better performance on complex sentences.
- Can learn patterns automatically from data.
- Can incorporate large-scale training information.
- Suitable for modern NLP applications.

Limitations

- Requires large training datasets.

- More computationally expensive.
- Less interpretable than traditional rule-based parsing.
- May reproduce biases present in training data.

3.4 Comparison

Aspect	CFG	PCFG	Neural Parsing
Represents syntax	Excellent	Excellent	Excellent
Handles ambiguity	Represents alternatives	Ranks alternatives	Learns contextual interpretation
Probability	No	Yes	Yes/learned scores
Context understanding	Low	Moderate	High
Training data	Grammar rules	Treebank/statistics	Usually large datasets
Interpretability	High	High	Lower
Computational requirements	Low/Medium	Medium	High
Complex real-world sentences	Limited	Better	Strong
Adaptability	Low	Moderate	High

3.5 Critical Analysis

CFG is useful for explaining grammatical structures but cannot effectively choose between competing interpretations.

PCFG improves CFG by assigning probabilities and selecting the most likely parse.

Neural parsing goes further by using contextual representations and learned patterns.

Justification

For a modern real-world NLP system, neural parsing is generally the strongest choice among the three when sufficient training data and computational resources are available.

However, a hybrid architecture can be even more practical:

Neural model → contextual scoring

PCFG/grammar constraints → syntactic validation

This combines statistical strength with grammatical control.

Conclusion

CFG is best for basic structural representation.

PCFG is better for probabilistic ambiguity resolution.

Neural parsing is generally more effective for complex real-world language because it incorporates context and learns from data.

Therefore, neural parsing, potentially combined with symbolic grammar constraints, is the preferred solution for real-world applications.

4. Feature Structures vs Subcategorization Frames

Introduction

A grammar system needs to handle two important requirements:

1. **Subject–verb agreement**
2. **Verb argument structures**

Feature structures and subcategorization frames address these requirements, but they solve different problems.

4.1 Feature Structures

Feature structures represent grammatical properties using features and values.

Example:

Noun:

NUMBER = singular

PERSON = third

and:

Verb:

NUMBER = singular

PERSON = third

For:

The student reads a book.

The features are compatible:

student:

NUMBER = singular

reads:

NUMBER = singular

Therefore, the sentence satisfies subject–verb agreement.

For:

The student read books.

the system can also represent tense and other grammatical properties.

4.2 Feature Unification

Feature structures can be combined through unification.

For example:

Subject:

NUMBER = plural

PERSON = third

must agree with:

Verb:

NUMBER = plural

PERSON = third

If the features conflict:

Subject:

NUMBER = plural

Verb:

NUMBER = singular

the unification fails.

This provides a direct mechanism for enforcing grammatical consistency.

4.3 Subcategorization Frames

Subcategorization frames describe the types of arguments that a verb requires.

Examples:

Eat

eat → Subject + Object

Example:

The child eats food.

Give

give → Subject + Object + Recipient

Example:

The teacher gave the student a book.

Sleep

sleep → Subject

Example:

The baby sleeps.

Depend

depend → Subject + on + Object

Example:

Success depends on effort.

Subcategorization therefore describes what arguments a particular verb expects.

4.4 Comparison

Aspect	Feature Structures	Subcategorization Frames
Main purpose	Grammatical feature constraints	Verb argument requirements
Agreement	Excellent	Limited
Person	Yes	No
Number	Yes	No
Tense	Can represent	Not primary purpose
Verb arguments	Can encode some information	Excellent
Grammatical consistency	High	Moderate
Verb-specific constraints	Moderate	High
Unification	Yes	Usually not central
Example	Singular subject ↔ singular verb	Give → Subject + Object + Recipient

4.5 Critical Analysis

These techniques should not really be viewed as complete replacements for each other because they address different dimensions of grammar.

Feature structures answer:

“Are the grammatical features compatible?”

Subcategorization frames answer:

“What arguments does this verb require?”

For example:

“The doctors prescribe medicine.”

Feature structures can verify:

doctors = plural

prescribe = plural

Subcategorization can verify that:

prescribe → Subject + Object

Therefore, both techniques complement each other.

Justification

If the question specifically asks which provides better support for subject–verb agreement and grammatical correctness, feature structures are more suitable because they explicitly encode and unify grammatical features.

However, for verb argument structures, subcategorization frames are superior.

Best Industry Solution

Use both:

Feature Structures + Subcategorization Frames

This provides:

- agreement checking
- tense and person checking
- argument validation
- verb-specific constraints
- improved grammatical correctness

Conclusion

Feature structures are the better choice for enforcing grammatical agreement, while subcategorization frames are better for enforcing verb argument structures. Therefore, a combined approach provides the strongest grammar system.

5. Transition-Based vs Graph-Based Dependency Parsing

Introduction

Large-scale NLP systems need dependency parsers that provide:

- high speed
- good accuracy
- scalability
- efficient memory usage
- reliable dependency structures

Two major approaches are transition-based parsing and graph-based parsing.

5.1 Transition-Based Parsing

Transition-based parsing constructs a dependency tree incrementally using a sequence of decisions.

A typical parser maintains:

- Stack
- Buffer

- Set of dependency relations

Common transitions include:

SHIFT

LEFT-ARC

RIGHT-ARC

REDUCE

For example, the parser may process a sentence word by word and decide which dependency relation should be created next.

Advantages

- Very fast.
- Low computational complexity.
- Suitable for real-time systems.
- Requires relatively limited memory.
- Effective for large datasets.

Limitations

- Early errors can propagate.
- Decisions are often locally focused.
- Performance depends on the quality of the transition classifier.
- Long-distance relationships can sometimes be difficult.

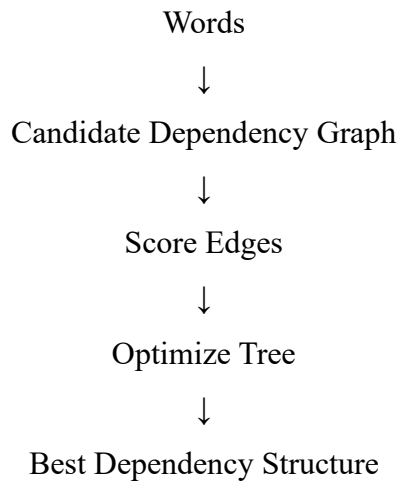
5.2 Graph-Based Parsing

Graph-based parsing treats dependency parsing as a graph optimization problem.

The parser:

1. Generates possible dependency relationships.
2. Assigns scores to candidate edges.
3. Searches for the highest-scoring valid dependency tree.

Conceptually:



Advantages

- Considers relationships globally.
- Less vulnerable to some forms of local decision error.
- Can provide high-quality dependency structures.
- Better suited to global structural optimization.

Limitations

- More computationally expensive.
- Requires greater memory.
- Can be slower for very large datasets.
- Global optimization increases processing requirements.

5.3 Comparison

Aspect	Transition-Based	Graph-Based
Decision method	Sequential/local transitions	Global optimization
Processing	Incremental	Sentence-level
Speed	Very high	Moderate
Memory	Lower	Higher
Error propagation	Higher	Lower
Global structure	Limited	Strong
Real-time processing	Excellent	Moderate
Large-scale processing	Very suitable	More resource-intensive
Accuracy	High	Often strong
Implementation	Relatively simple	More complex

5.4 Performance Analysis

For a large dataset containing millions of sentences, processing speed becomes an important factor.

Transition-based parsing can process sentences incrementally:

Input → Transition → Transition → Transition → Dependency Tree

This makes it computationally efficient.

Graph-based parsing may need to consider many possible dependency relationships before selecting the best tree.

Therefore:

Transition-based → faster

Graph-based → stronger global decision-making

5.5 Critical Analysis

The choice depends on the application's priorities.

If speed is the main requirement:

Choose transition-based parsing.

It is suitable for:

- search engines
- chatbots
- real-time assistants
- large-scale preprocessing
- streaming NLP

If global accuracy is the primary requirement:

Graph-based parsing can be preferable because it evaluates dependency structures more globally.

For example, in complex sentences with long-distance relationships, global scoring can help prevent some locally optimal but globally incorrect decisions.

5.6 Justification for Large-Scale NLP

The problem specifically states:

“A large-scale NLP system must perform fast and accurate dependency parsing on big datasets.”

Therefore, both speed and accuracy matter.

For large-scale applications, transition-based parsing is generally the more practical choice because:

- it is computationally efficient;
- it requires less memory;
- it processes sentences incrementally;

- it supports high throughput;
- it is suitable for real-time and large-scale pipelines.

A modern system can improve its accuracy by using a neural transition classifier and appropriate training data.

Conclusion

Transition-based parsing is more suitable when speed, scalability and throughput are major requirements. Graph-based parsing is preferable when global structural accuracy is more important than processing speed.

For the stated large-scale requirement, transition-based parsing is the recommended choice, potentially enhanced with neural scoring to achieve a better balance between speed and accuracy.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation

Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand
-------------------------	-----	--	-----------------------------------	---------------------------------------	--

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____